Introducción

APIS gráficas

Bibliotecas:

- OpenGL estandard desde 1990.
- DirectX biblioteca de Microsoft.
- Vulkan alternativa moderna a OpenGL y DirectX.

Motores Gráficos:

- Unity 3D.
- Unreal.
- Godot.

OpenGL vs Motores gráficos

Un programa que usa OpenGL contiene código para dibujar cada frame.

Este código se puede ejecutar en la CPU, enviando el resultado de cada instrucción de dibujo a la GPU, o en la GPU. En este último caso se envía a la GPU una descripción de la escena, que solo se debe reenviar cada vez que se cambia la escena.

En un motor gráfico se define (con un editor interactivo y/o código) la escena y el comportamiento de los objetos de la escena. El motor se encarga de generar el código para visualizar cada frame.

Motores gráficos

Incluyen:

- Gestión de escenas, cámaras, luces y materiales.
- Simulación física, animación y colisiones.
- Importación de modelos y texturas.
- Interacción, interfaz de usuario y sonido.

Godot Engine

Godot Engine

Godot es un motor libre y de código abierto para el desarrollo de aplicaciones gráficas interactivas.

- Entorno de desarrollo amigable y multiplataforma
- Permite la creación de experiencias 2D y 3D
- Exportación a múltiples plataformas (PC, Web, móviles)
- Se programa en GDScript (similar a Python)
- Comunidad activa y creciente
- Gran cantidad de recursos educativos y ejemplos disponibles

Instalación

Descarga

- `https://godotengine.org/download`
- Disponible para: Linux, Mac y Windows
- No requiere instalación, basta con ejecutar el binario
- Manual en `https://docs.godotengine.org`

Godot funciona sin configuración adicional incluso en equipos modestos o en aulas con usuarios sin permisos de administrador.

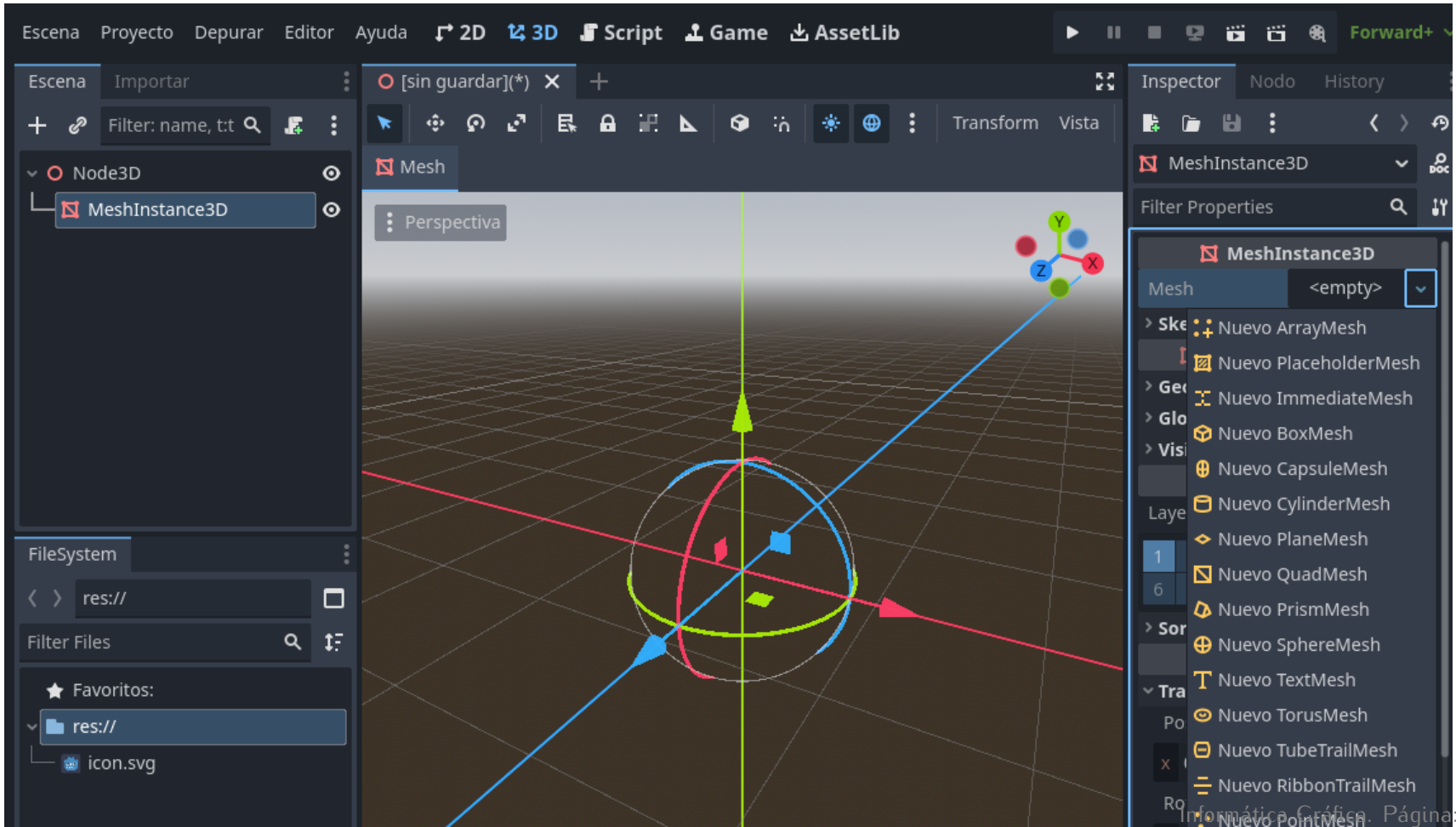
Licencia

- MIT (libre y sin restricciones comerciales)

Creación de objetos

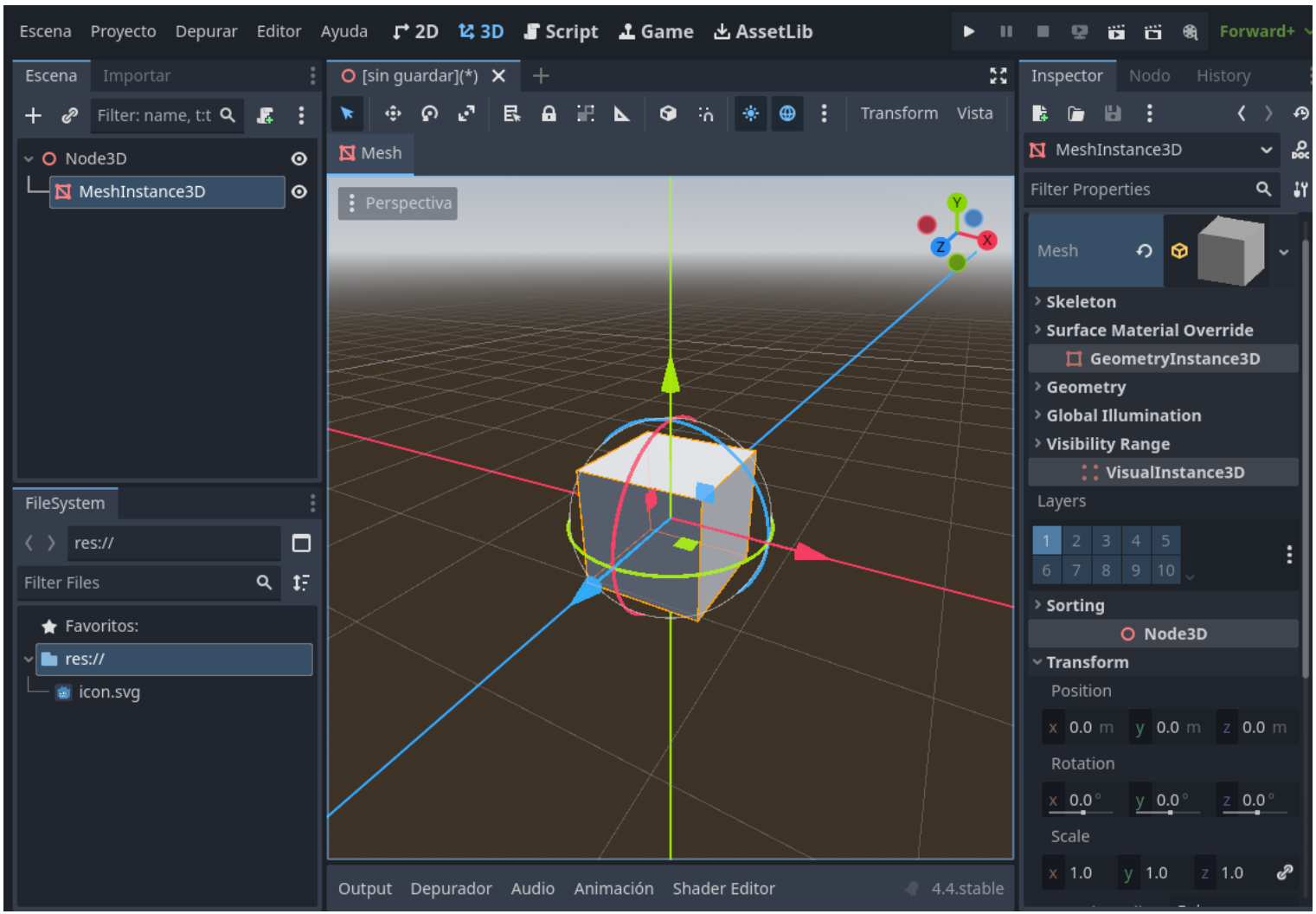
Nodos **MeshInstance3D**. Propiedad Mesh.

- Primitivas 3D: cubo, esfera, cápsula, etc.
- Propiedades editables desde el panel Inspector



Transformaciones geométricas

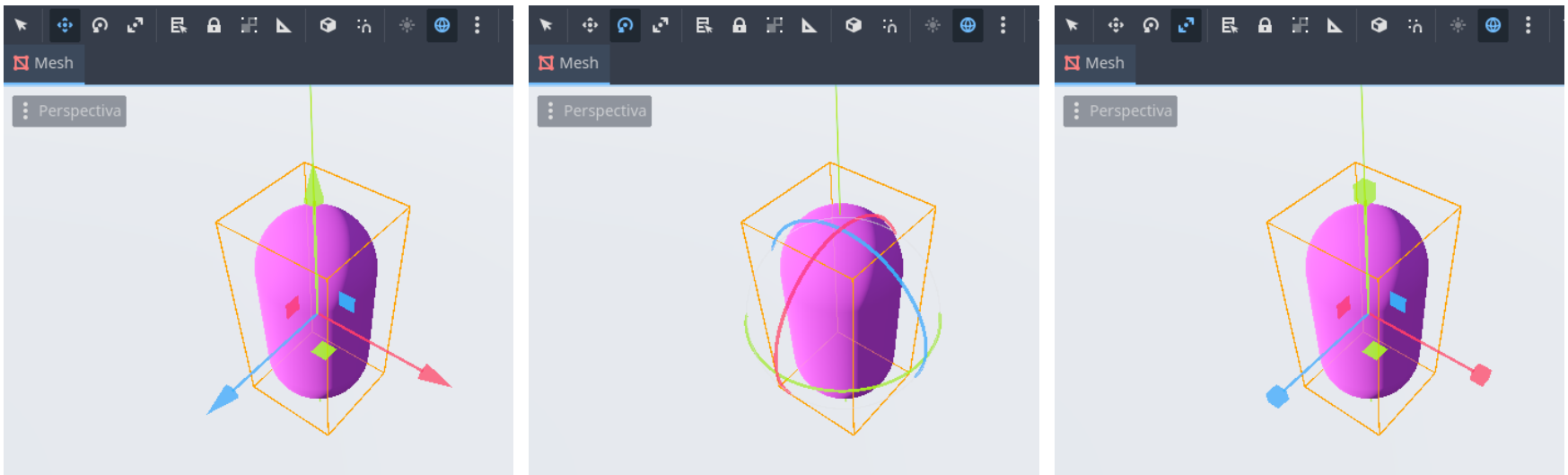
Para transformar un objeto, se puede modificar su posición, rotación o escala desde el panel Inspector.



Modificación interactiva de objetos

Al seleccionar un objeto, se puede manipular directamente en el editor usando los controles de:

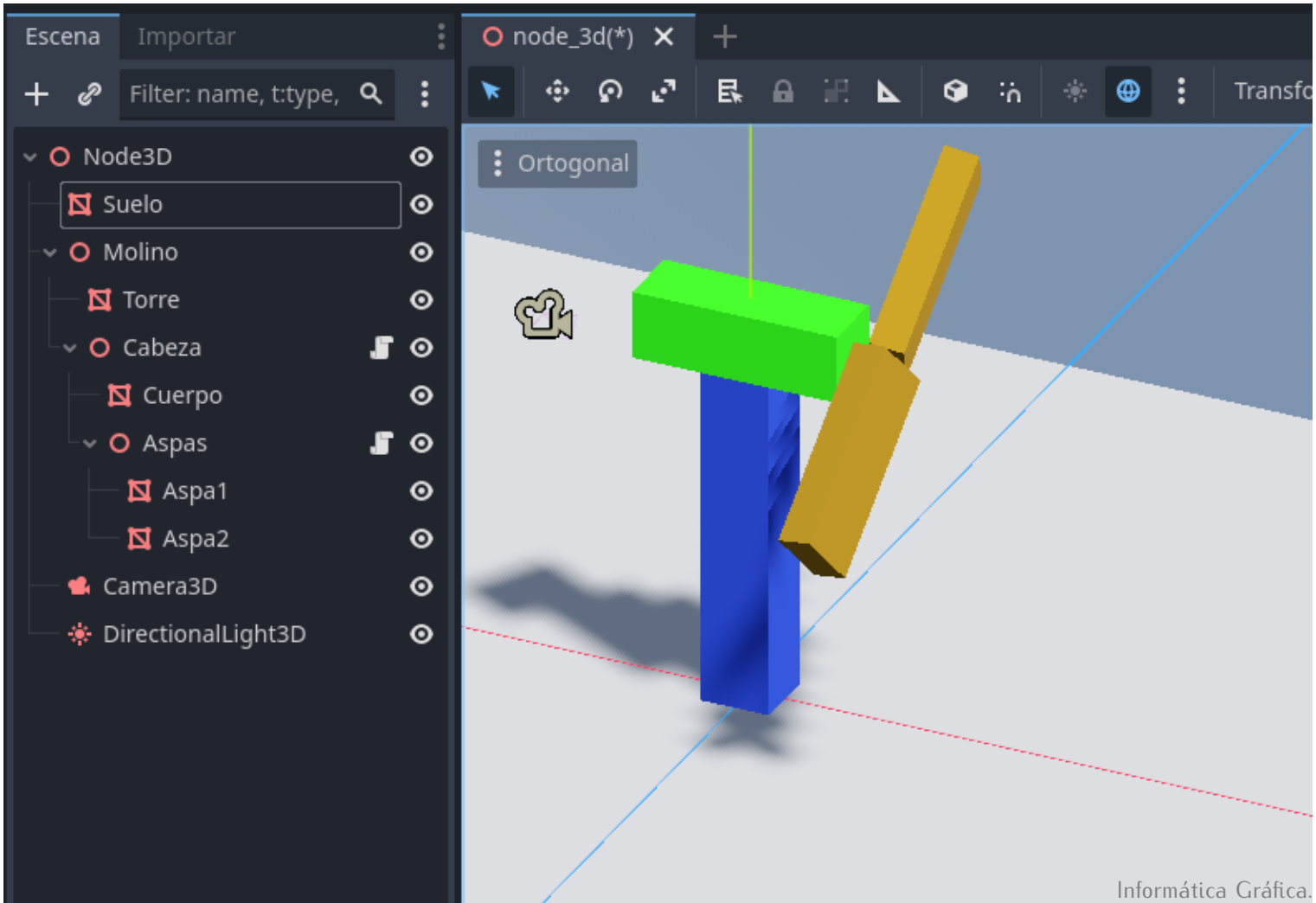
- Posición
- Rotación
- Escalado



Escena en Godot

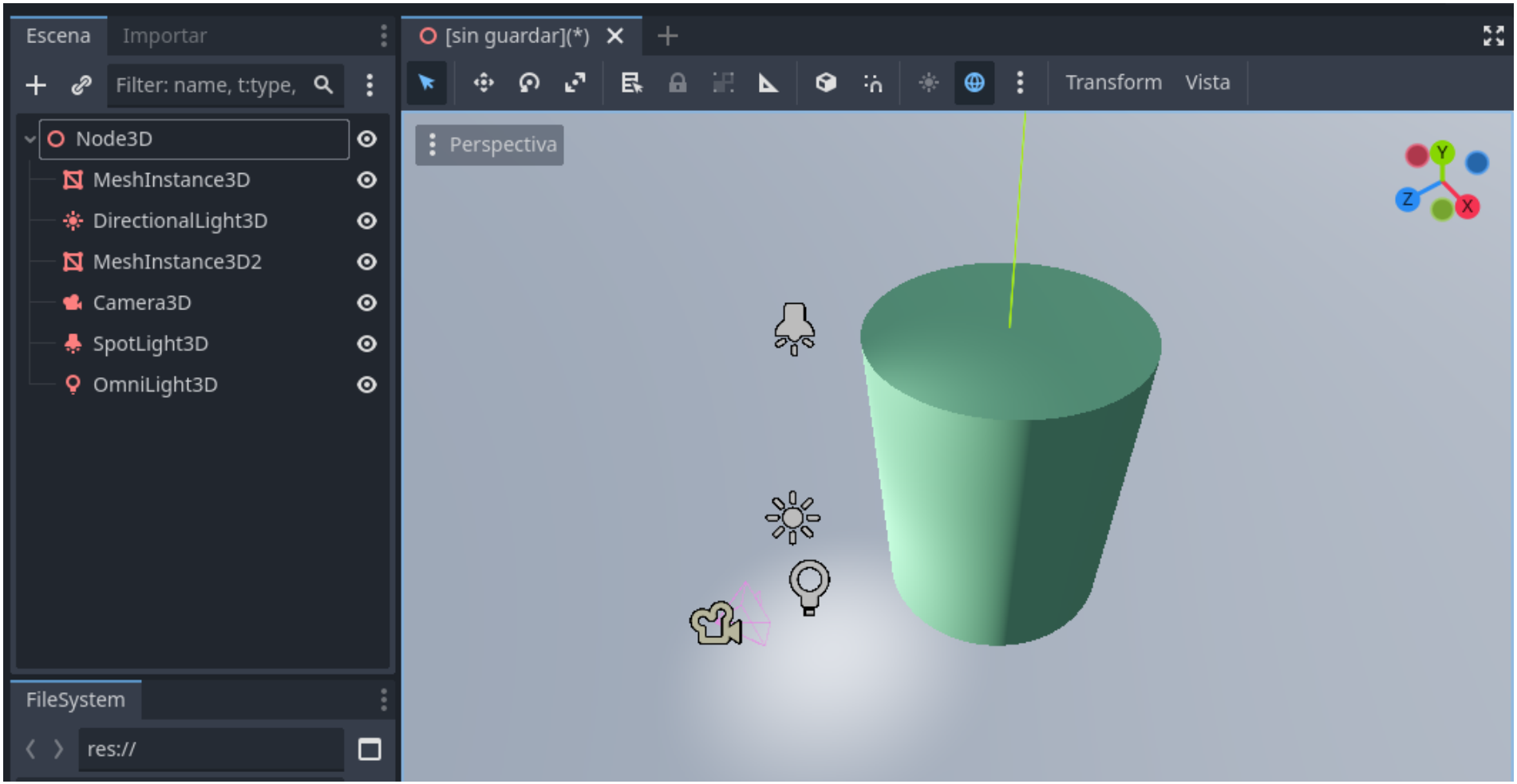
El panel de escena contiene la jerarquía de nodos de la escena actual.

Los nodos heredan propiedades de sus padres.



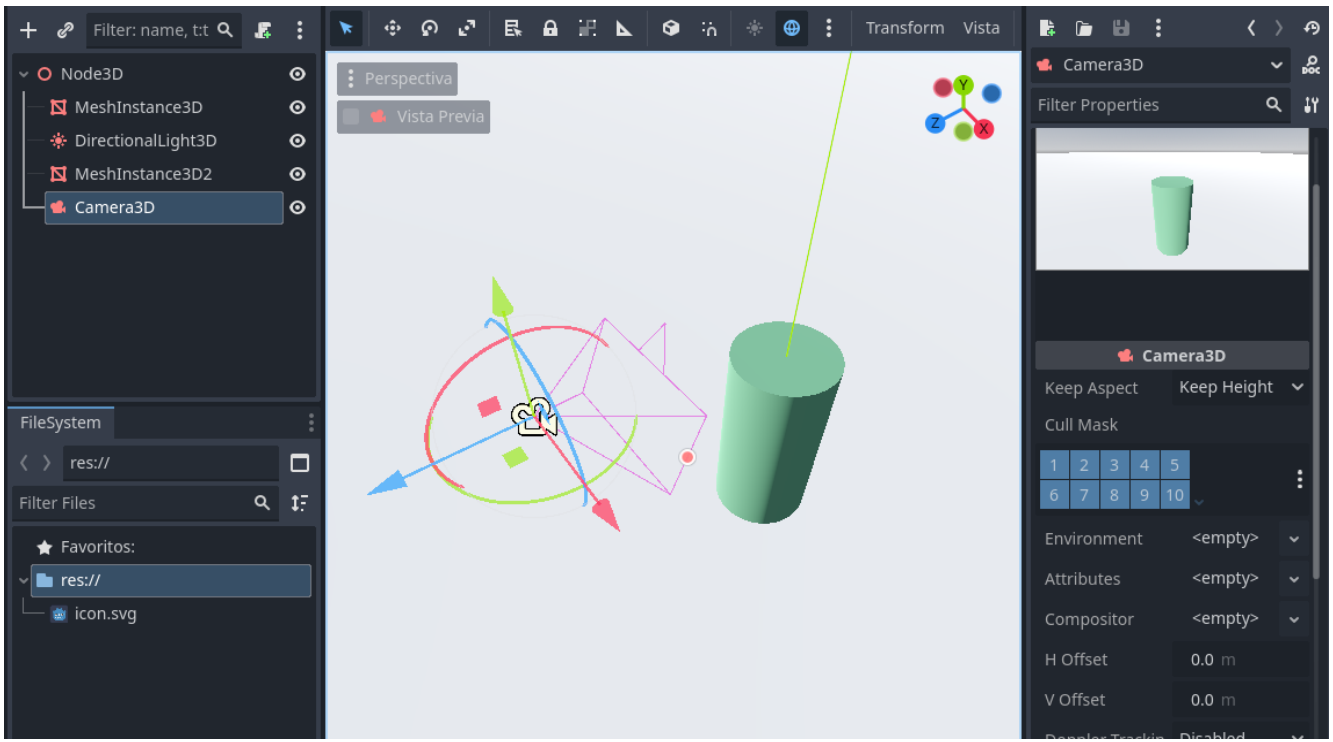
Cámara y luces en la escena

Las cámaras y luces son nodos que se añaden como cualquier otro objeto en la escena.



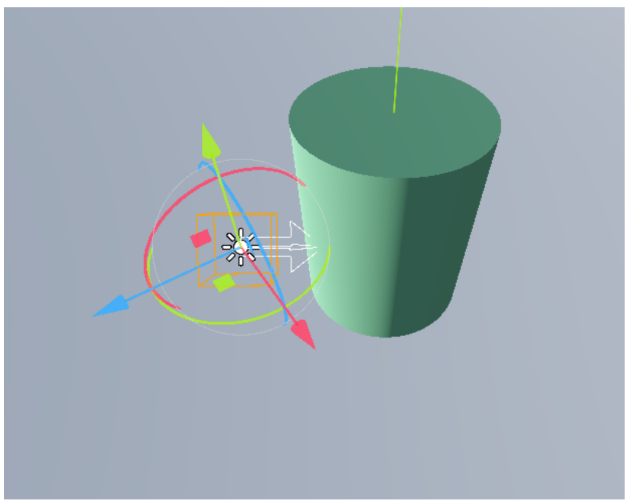
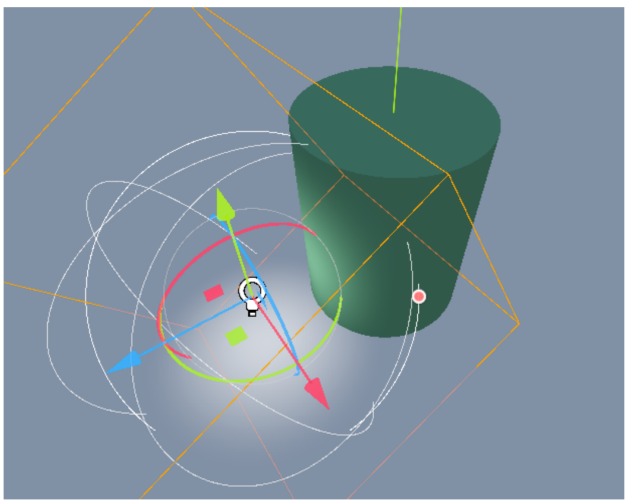
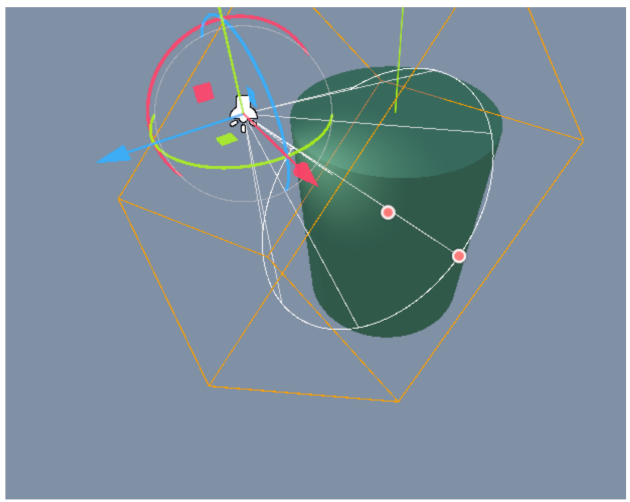
Cámara

- Al menos una cámara debe estar presente en la escena para renderizar.
- La posición y orientación se puede ajustar interactivamente.
- Los parámetros se cambian en el Inspector.



Luces

- Se pueden añadir diferentes tipos de luces:
  - Direccional
  - Puntual
  - Foco
- Las propiedades se editan desde el panel Inspector.

Interacción

En Godot se pueden asociar scripts a los nodos. Cada script puede definir funciones especiales:

- `_ready()`: se ejecuta una sola vez antes del prinmer frame (cuando el nodo está listo en la escena).
- `_process(delta)`: se ejecuta en cada frame.

```
extends Node3D

@export var speed := 2.0  # Velocidad en unidades por segundo

func _ready():
        print("Posicion inicial:", global_position)

func _process(delta):
        translate(Vector3(speed * delta, 0, 0)) # Mover
        print("Posicion actual:", global_position)
```

Interacción

Se puede procesar los eventos de entrada del usuario mediante funciones especiales de Godot.

```
extends Node3D

func _input(event):
        if event.is_action_pressed("ui_cancel"):
                get_tree().quit()
```

Ejemplo básico

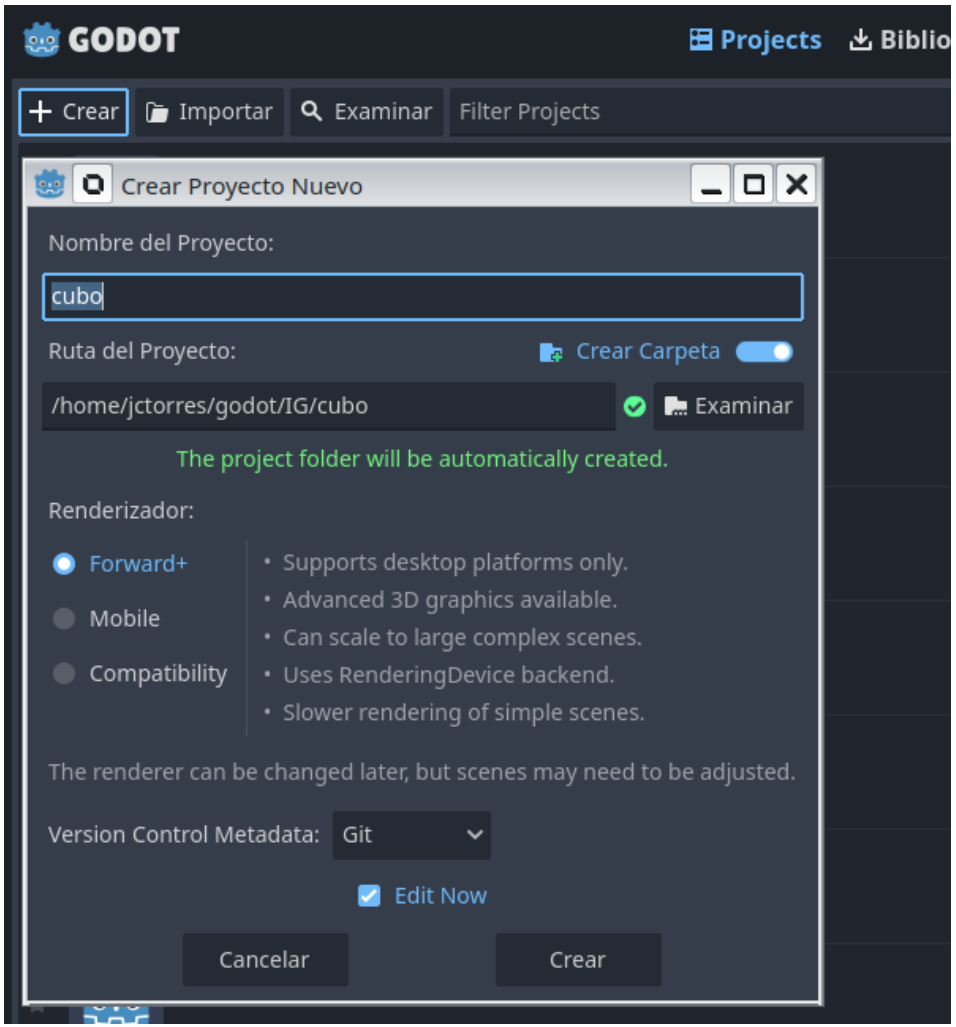
Resumen:

1. Crea un nuevo proyecto en Godot.
2. Crea una escena 3D.
3. Crea un cubo en la escena (MeshInstance3D con CubeMesh).
4. Cambia su escala desde el Inspector a (10,10,10).
5. Asignale un material y cambia su color.
6. Crea una cámara y ajustala para que enfoque al cubo.
7. Crea una luz y situala en la escena.
8. Crea un nuevo script para que rote el cubo.
9. Ejecuta la escena.

Ejemplo básico

1. Crea un nuevo proyecto en Godot.

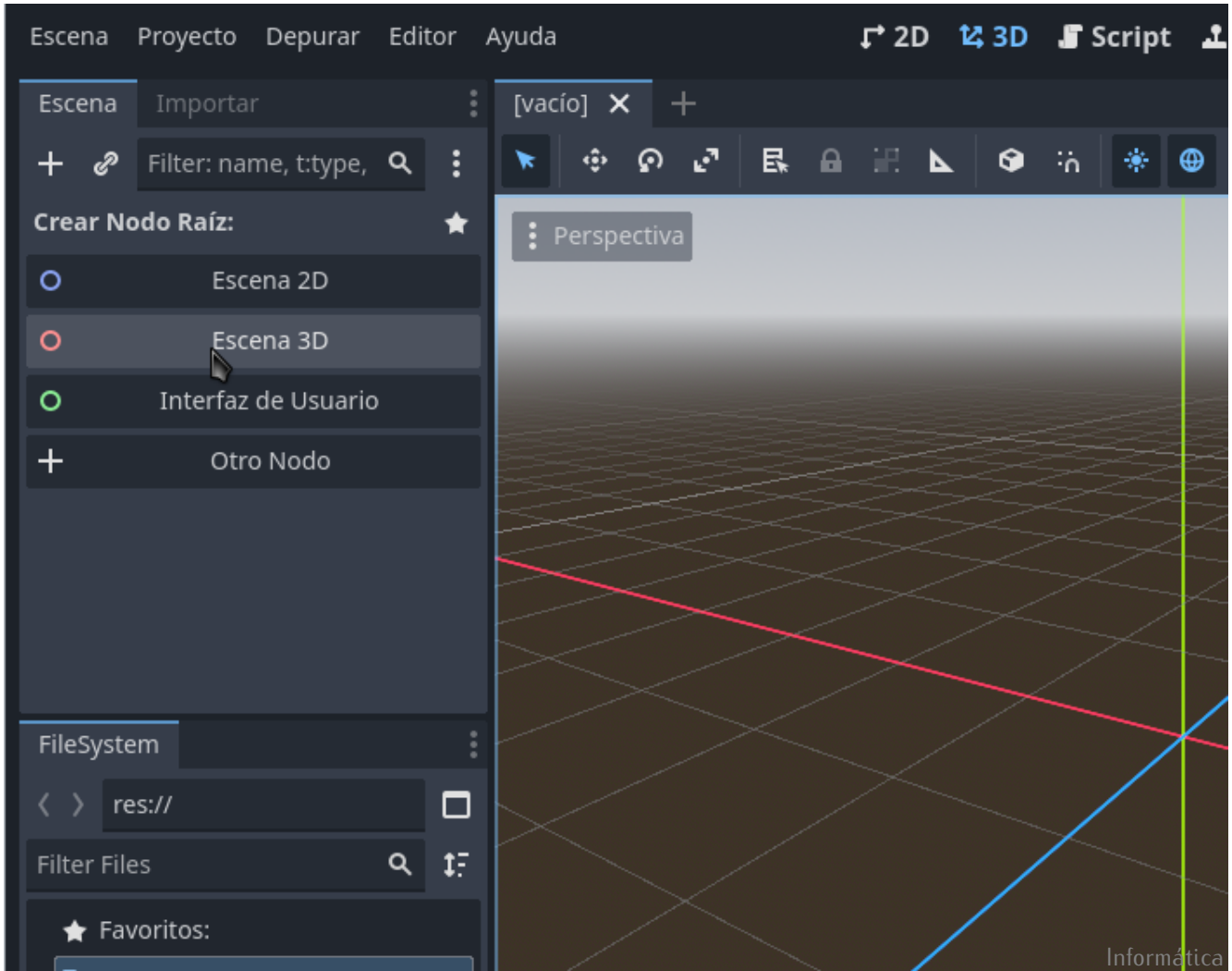
En el pnel de proyecto pulsa eb crear y elige la ubicación donde se guardará.



Ejemplo básico

2 Crea una escena 3D

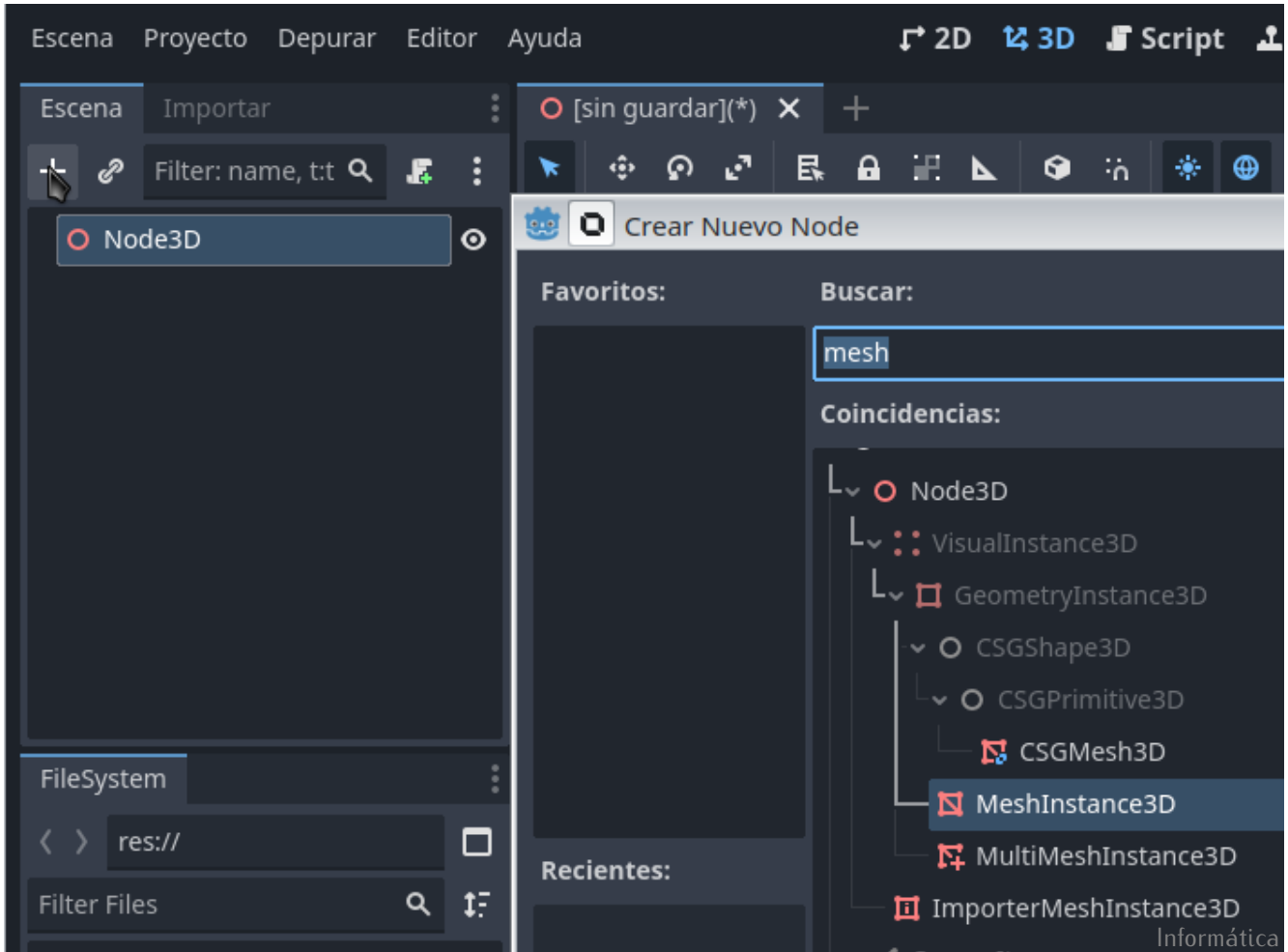
Selecciona Escena 3D en el menú de selección del nodo raíz.



Ejemplo básico

3. Crea un cubo en la escena (MeshInstance3D con CubeMesh).

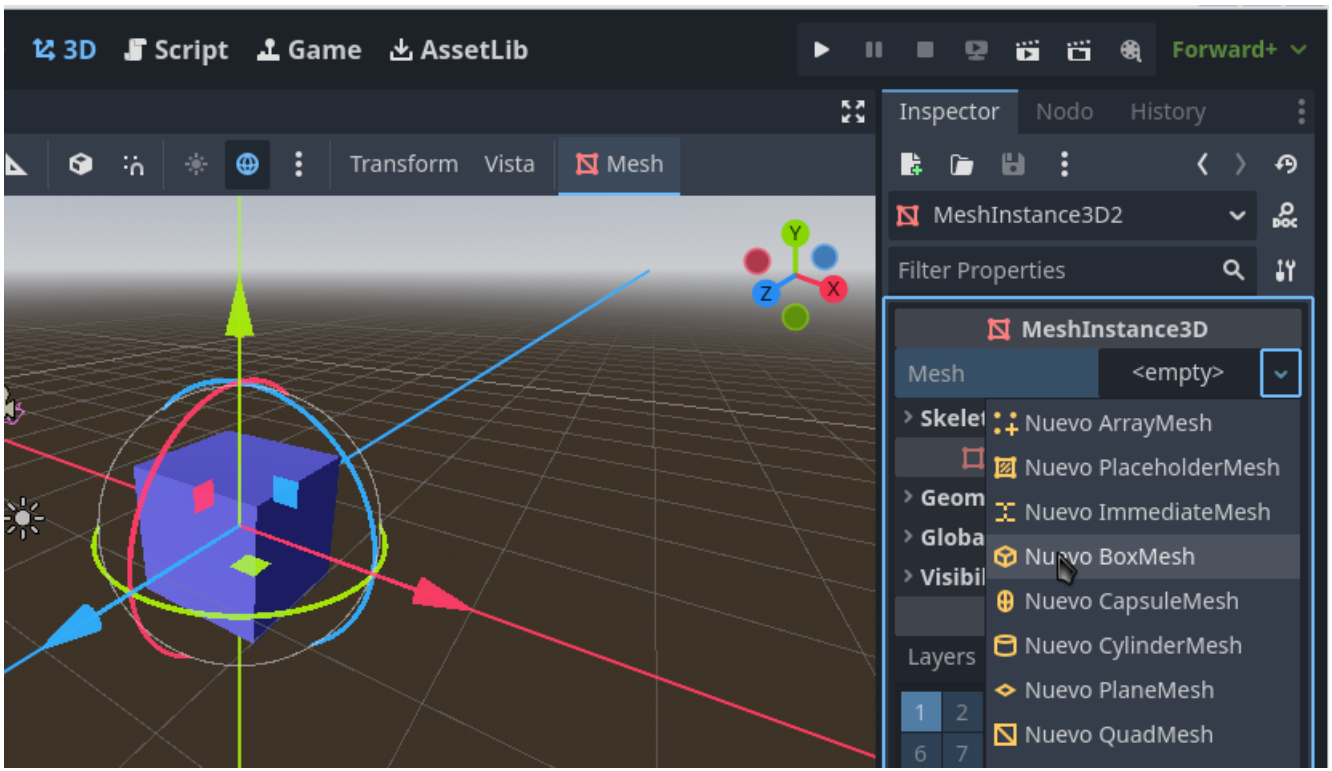
Añadimos un nuevo nodo (pulsando en +), y seleccionamos el tipo MeshInstance3D.



Ejemplo básico

3. Crea un cubo en la escena (MeshInstance3D con CubeMesh).

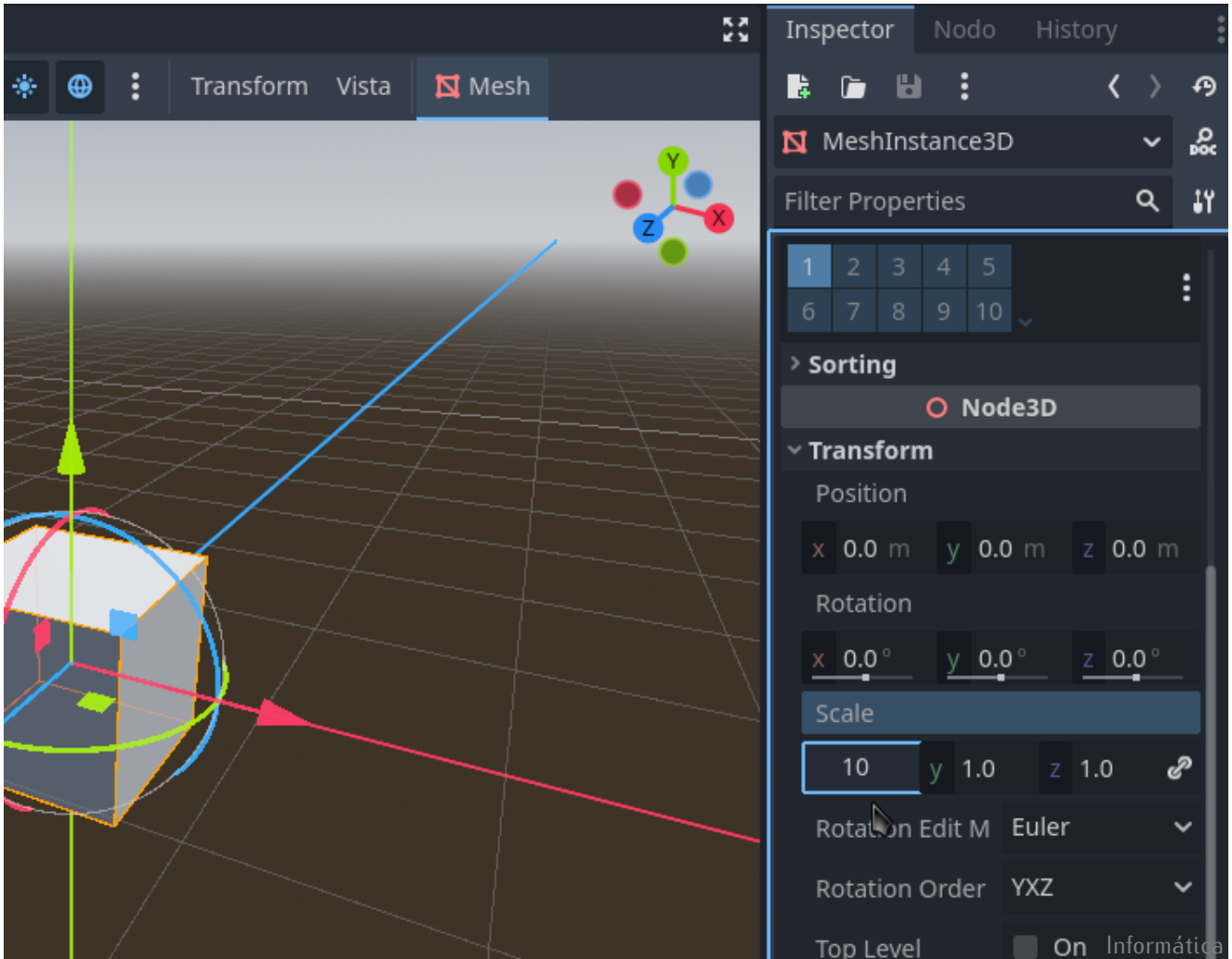
Con el nuevo nodo seleccionado en el panel inspector seleccionamos como malla (Mesh) **Nuevo BoxMesh**.



Ejemplo básico

4 Cambiar tamaño del cubo.

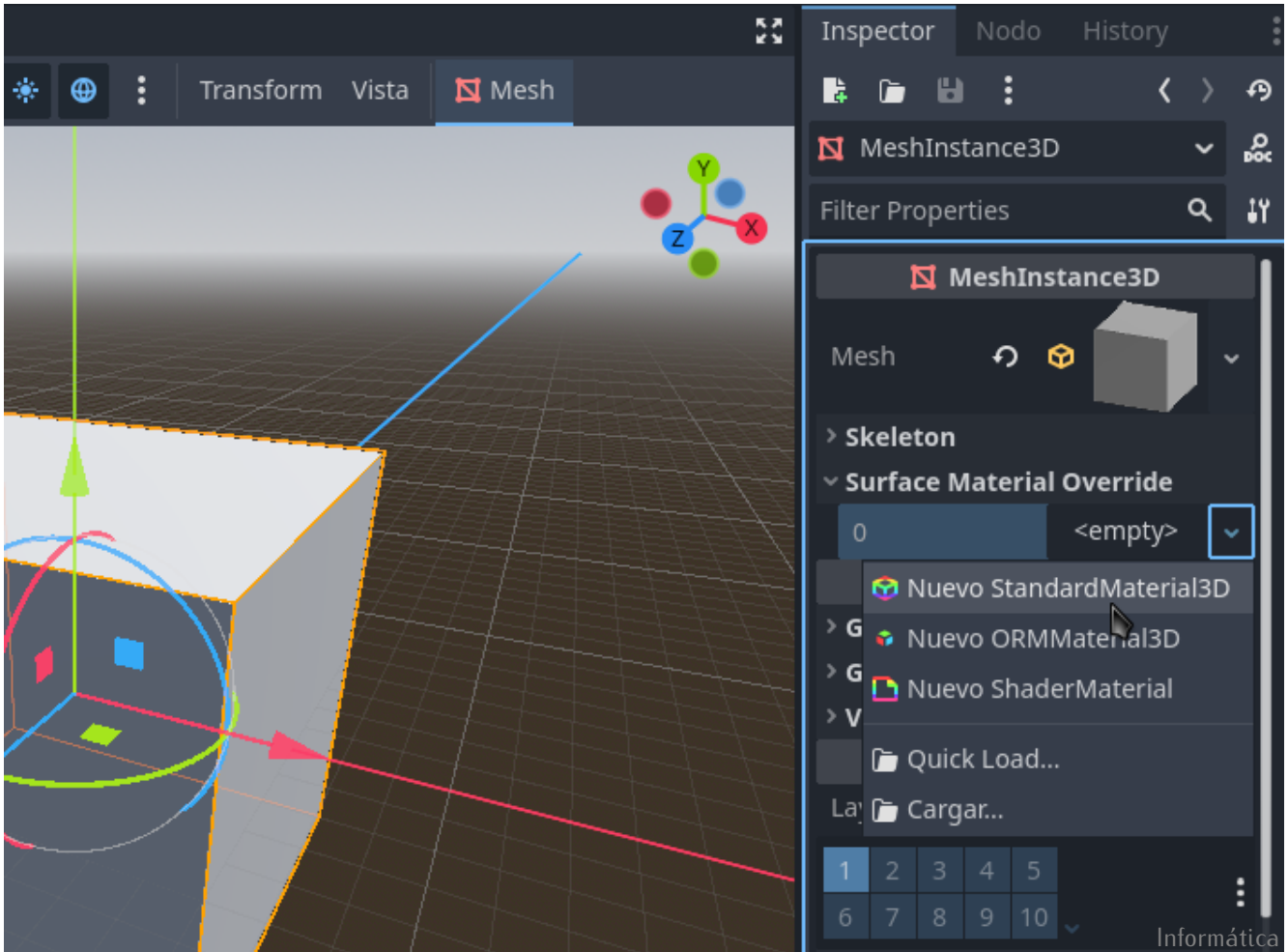
Busca en el inspector el bloque transform, ábrelo y modifica el valor de la escala.



Ejemplo básico

5 Asignale un material y cambia su color.

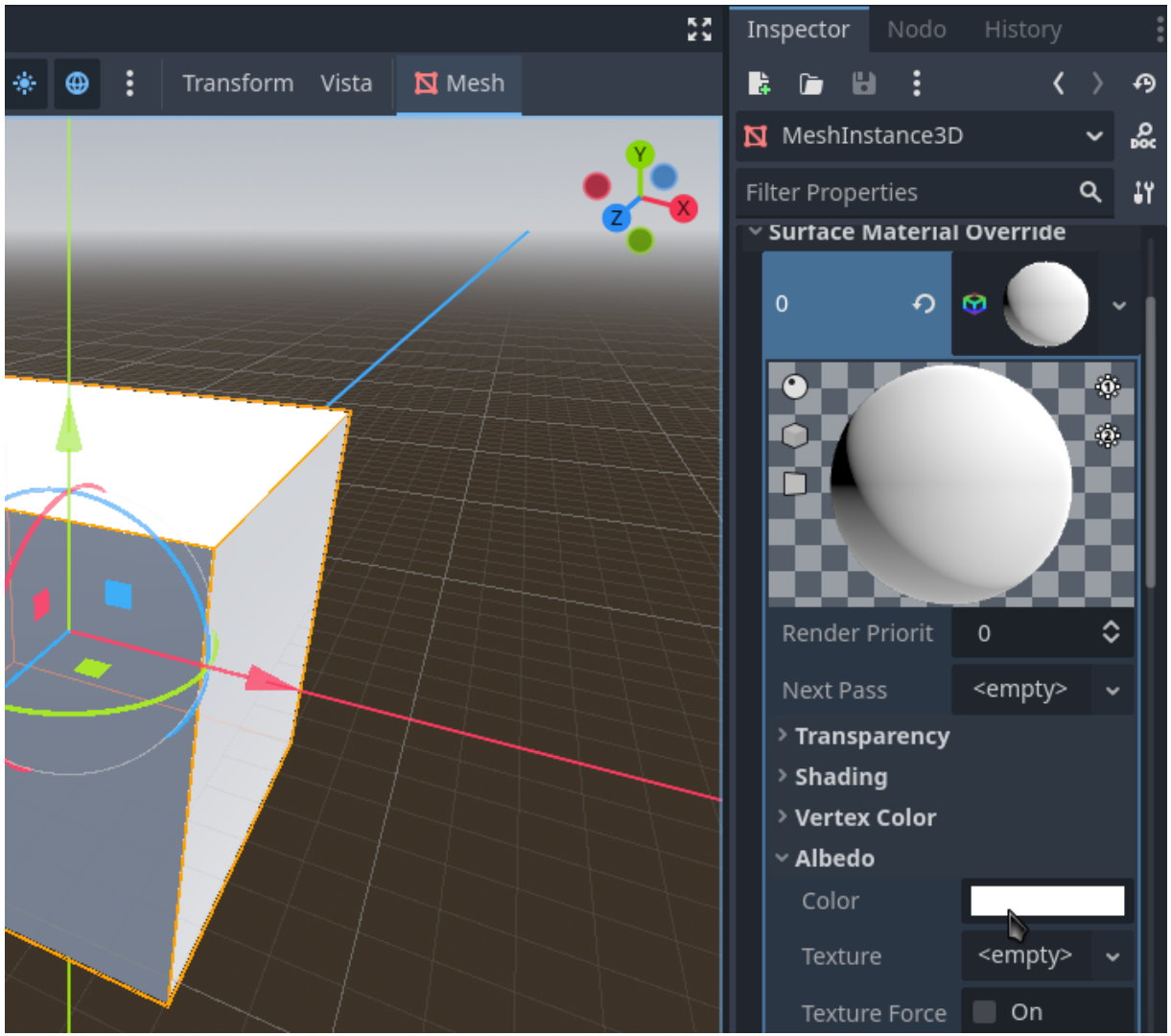
Busca el atributo **Surface Material Override** y asignale el valor **Nuevo StandardMaterial3D**.



Ejemplo básico

5. Asignale un material y cambia su color.

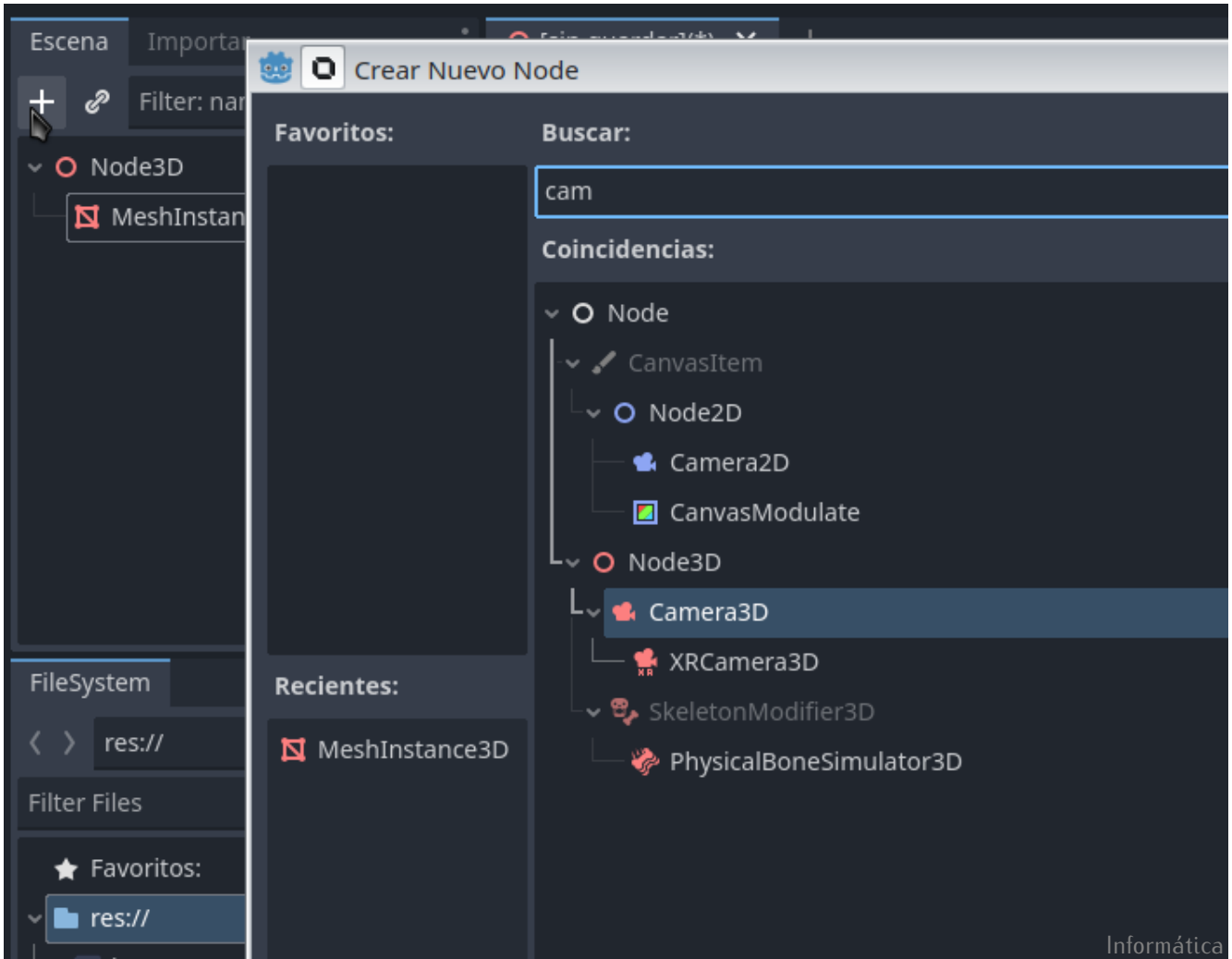
Pulsa en la esfera que ha aparecido en **Surface Material Override** y entre los atributos que aparecen busca **Albedo**. Expande el grupo, pulsa en el color y modificalo.



Ejemplo básico

6 Crea una cámara y ajustala para que enfoque al cubo.

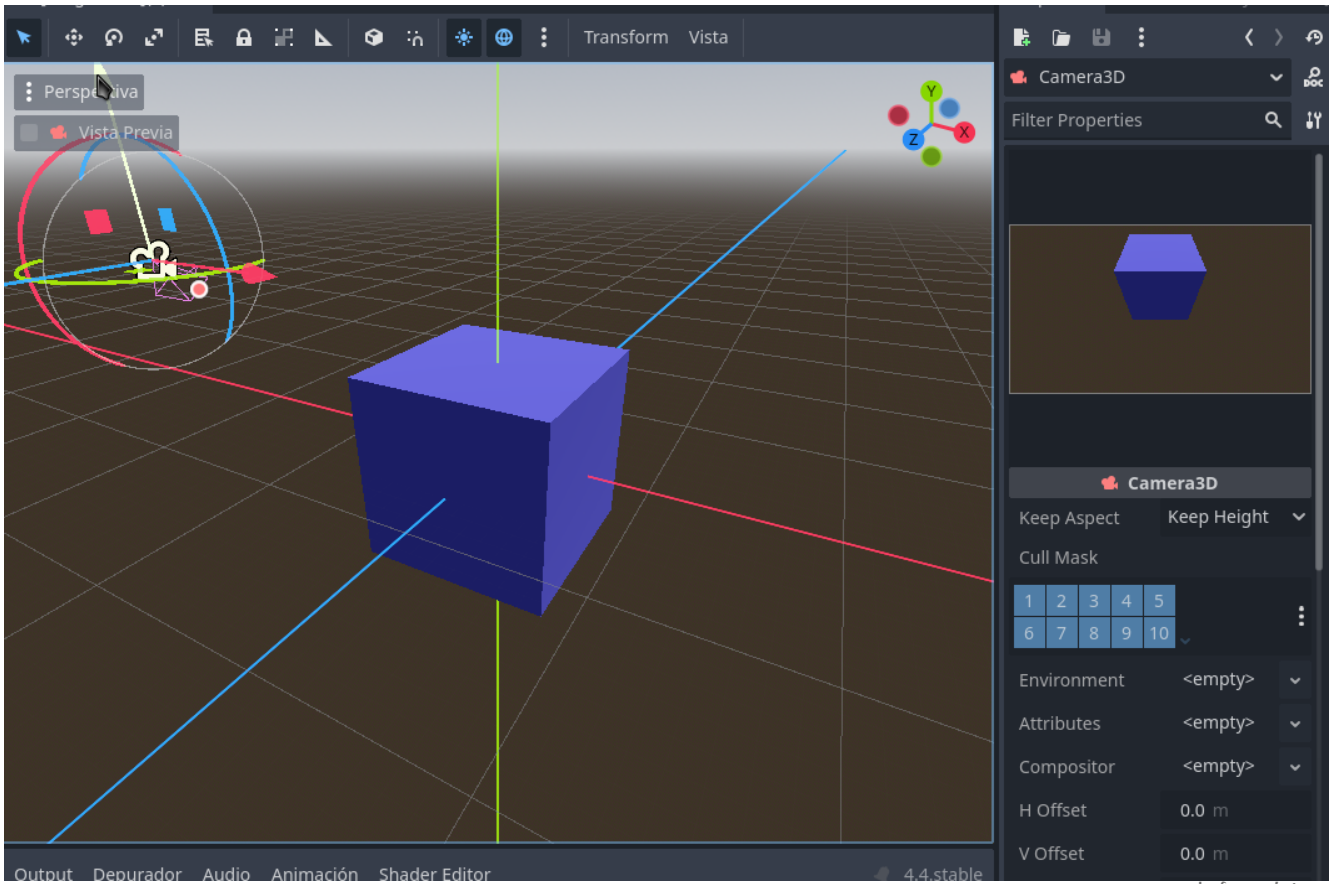
Añade un nuevo nodo (pulsando en +), y seleccionamos el tipo **Camera3D**.



Ejemplo básico

6 Crea una cámara y ajustala para que enfoque al cubo.

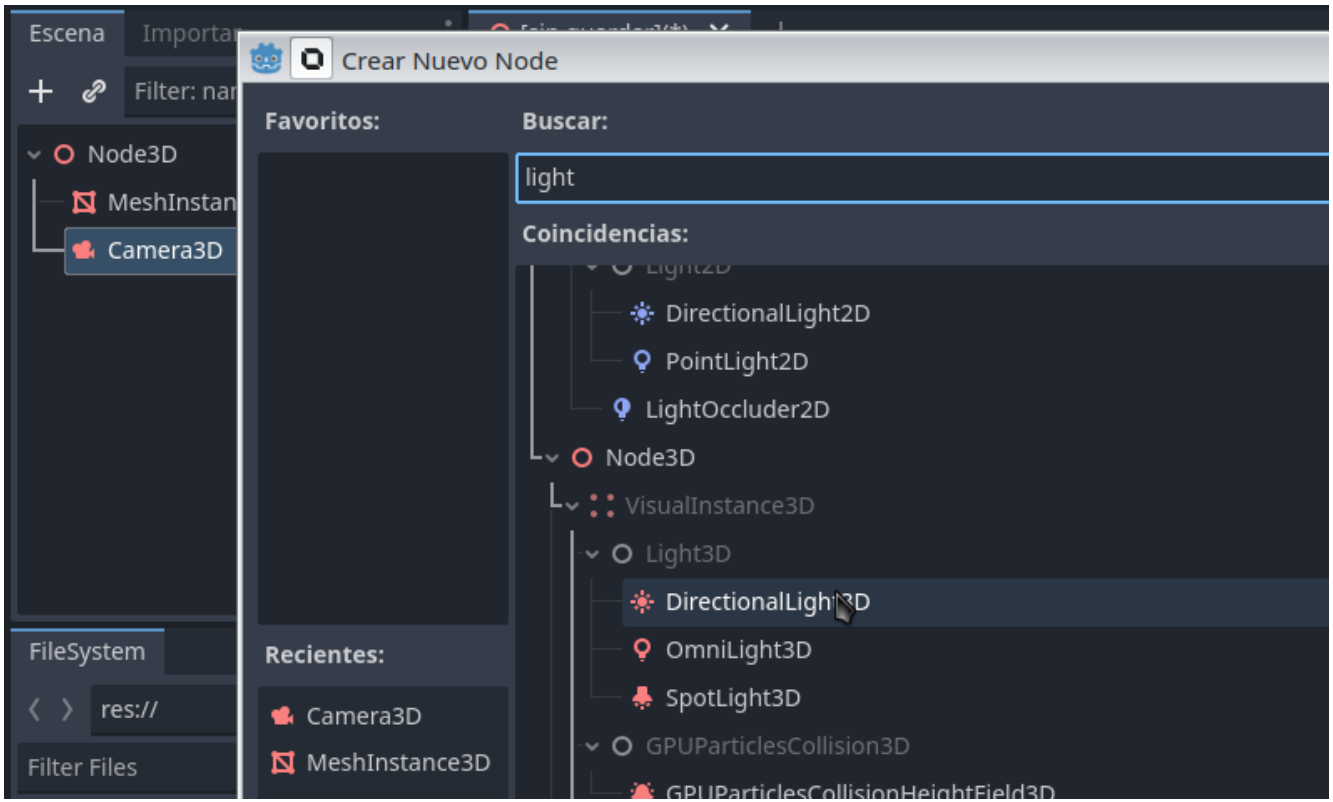
Utiliza los controles de transformación de la cámara para posicionarla para que vea todo el cubo (En el panel Inspector de la cámara se muestra la imagen que captura la cámara).



Ejemplo básico

7 Crea una luz y situala en la escena.

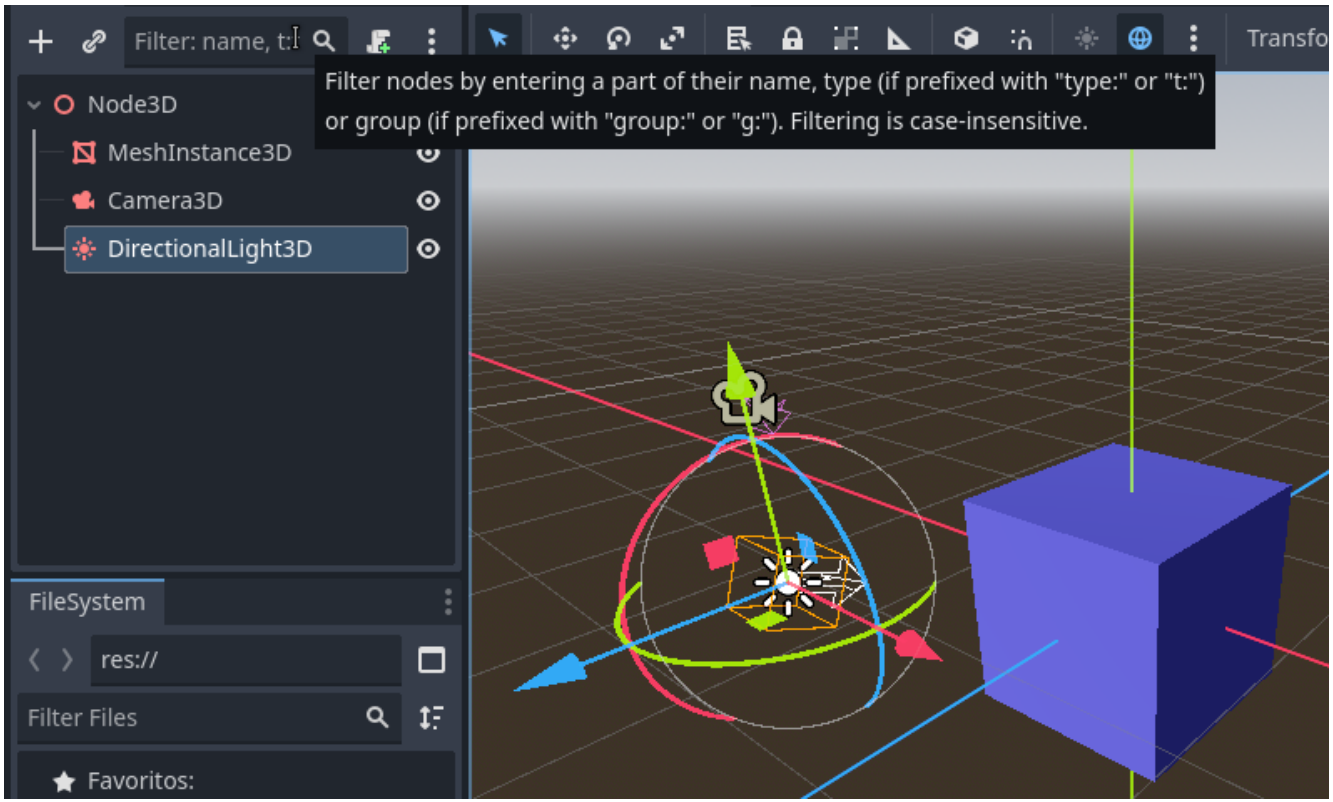
Añade un nuevo nodo (pulsando en +), y seleccionamos el tipo **DirectionalLight3D**.



Ejemplo básico

7 Crea una luz y situala en la escena.

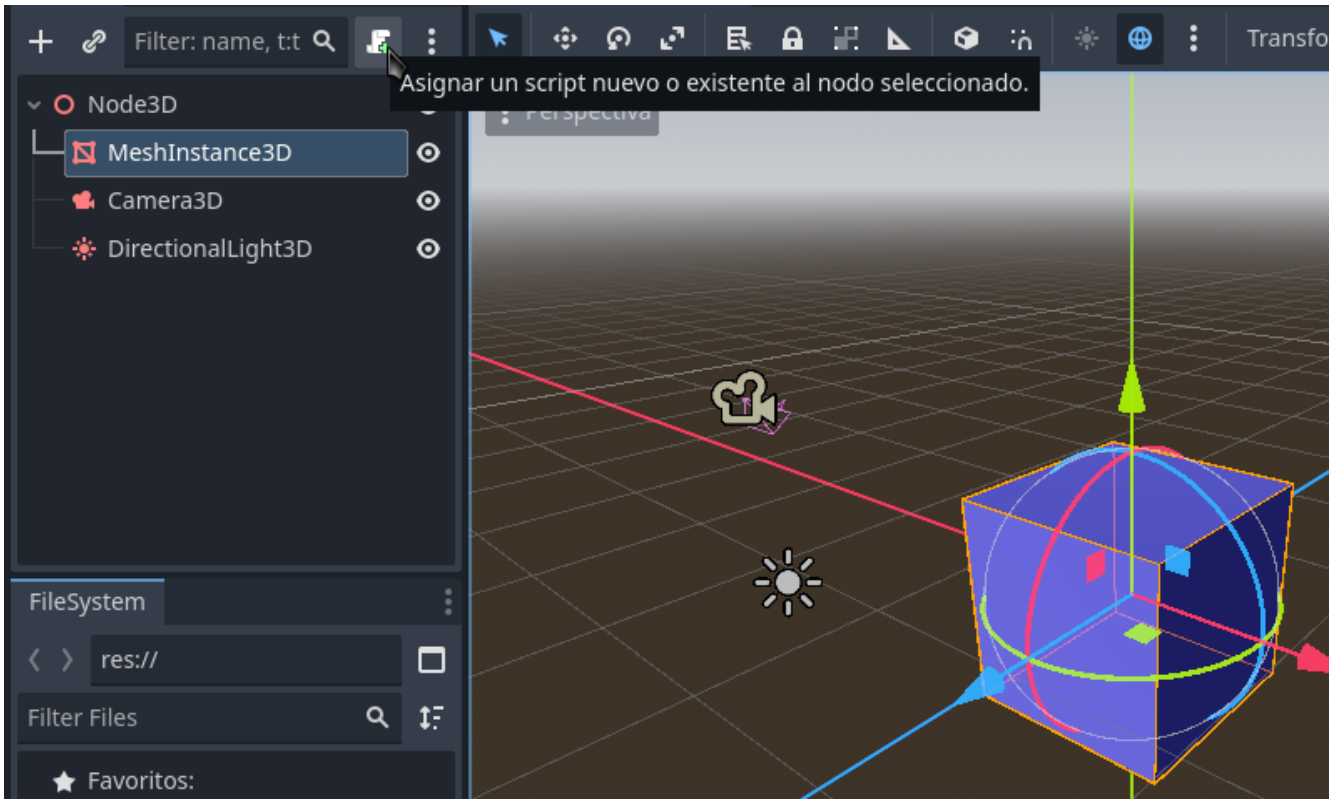
Posiciona y orienta la luz para que ilumine el cubo (La flecha blanca indica la dirección en la que está iluminando).



Ejemplo básico

8 Crea un nuevo script para que rote el cubo.

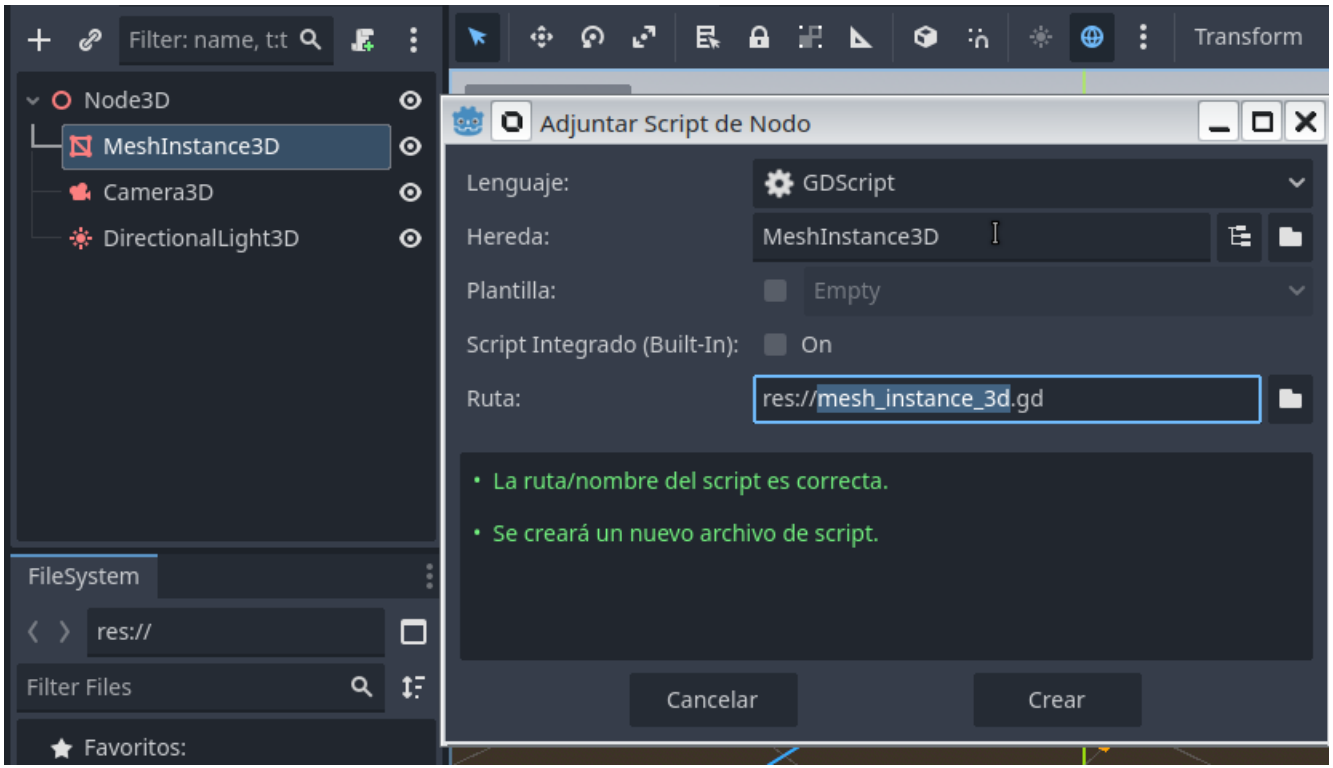
Pulsa en el icono .ªñadir script"teniendo seleccionado el cubo. Se creará un archivo de código vinculado a este nodo.



Ejemplo básico

8 Crea un nuevo script para que rote el cubo.

Asignale nombre al script en el panel emergente.



Ejemplo básico

8. Crea un nuevo script para que rote el cubo.

El panel central cambiará de modo 3D a script, mostrando un editor de texto. Escribe el siguiente código:

```
extends MeshInstance3D
@export var angle := 1.0

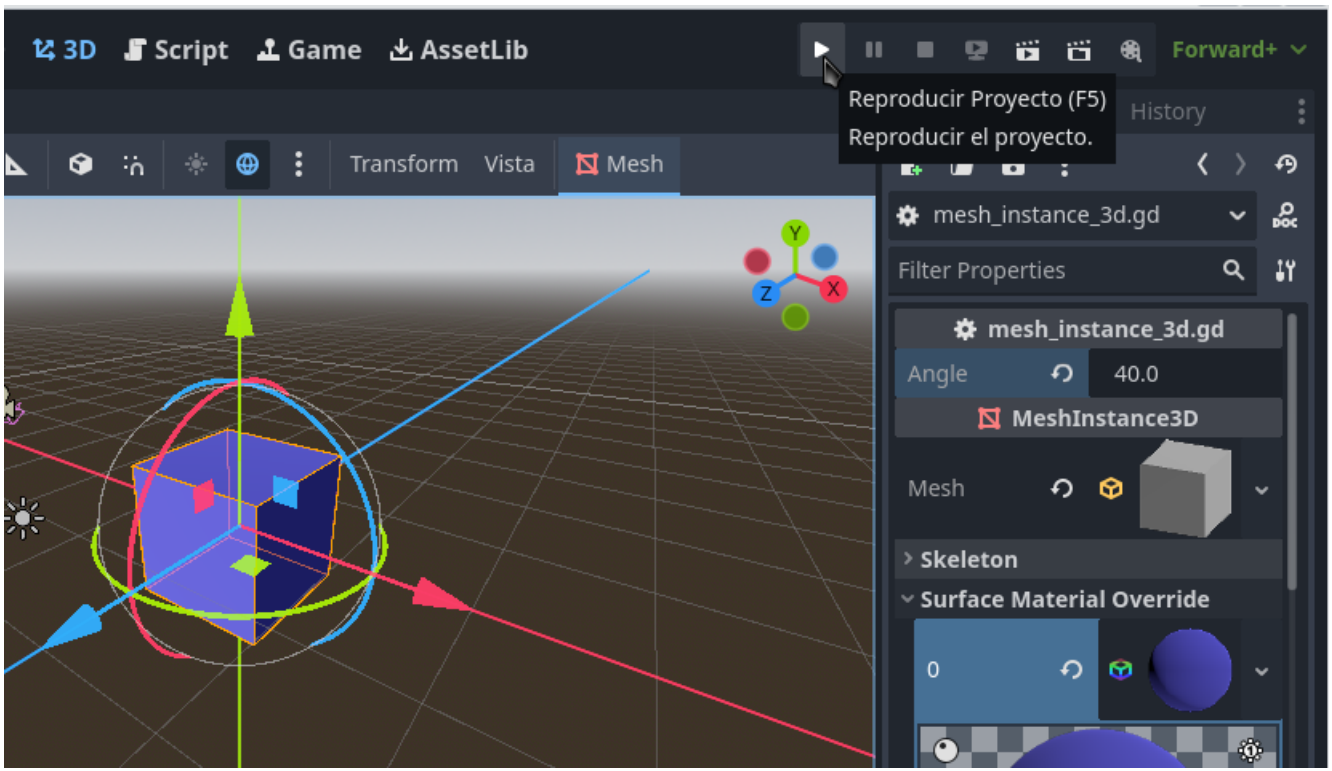
func _process(delta):
        rotate_y(deg_to_rad(angle*delta))
```

Ejemplo básico

9 Ejecuta la escena.

Antes de ejecutar el proyecto, salva la escena (Escena > Guardar Escena, o Ctrl.S)

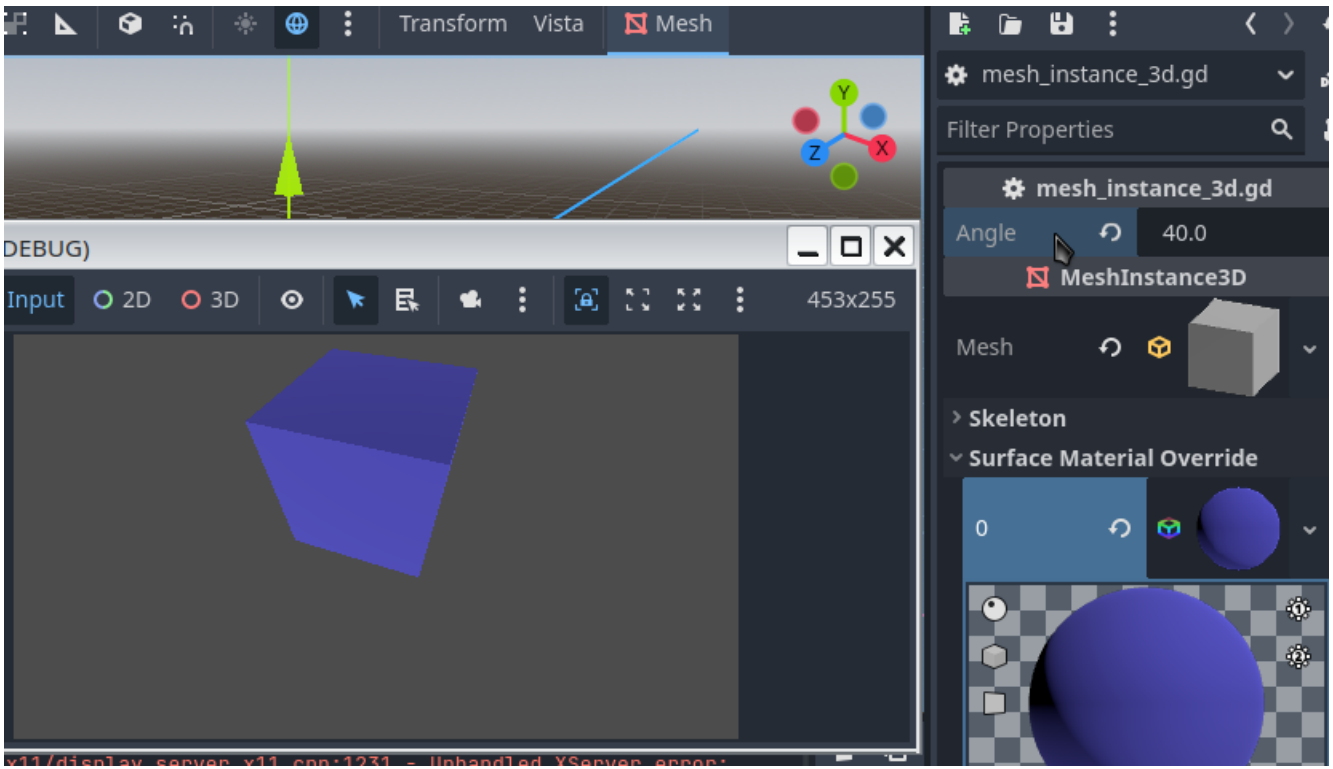
Pulsa el icono Reproducir Proyecto (o pulsa F5). El proyecto se ejecutará en una ventana nueva.



Ejemplo básico

9 Ejecuta la escena.

Mientras se ejecuta el proyecto puedes variar el valor de la propiedad angle del script que aparece como atributo de nuestro cubo en el panel inspector.



Crear geometría: Creación de un prisma triangular

```
extends MeshInstance3D
func _ready():
        var st = SurfaceTool.new()
        st.begin(Mesh.PRIMITIVE_TRIANGLES)
        var h = 1.0      # Altura del prisma
        var p0 = Vector3(0, 0, 1) # Vertices base
        var p1 = Vector3(1, 0, -1)
        var p2 = Vector3(-1, 0, -1)
        var p3 = p0 + Vector3(0, h, 0) # Vertices tapa
        var p4 = p1 + Vector3(0, h, 0)
        var p5 = p2 + Vector3(0, h, 0)
        _add_triangle(st, p0, p1, p2)   # Base inferior
        _add_triangle(st, p5, p4, p3)   # Base superior
        _add_triangle(st, p0, p4, p1)   # Cara lateral 1
        _add_triangle(st, p0, p3, p4)
        _add_triangle(st, p1, p5, p2)   # Cara lateral 2
        _add_triangle(st, p1, p4, p5)
        _add_triangle(st, p2, p3, p0)   # Cara lateral 3
        _add_triangle(st, p2, p5, p3)
        var mat = StandardMaterial3D.new() # material simple
        mat.albedo_color = Color(0.2, 0.6, 1.0)   # azul claro
        material_override = mat #Asigna material
        mesh = st.commit()       # Asignar malla
```

Crear geometría: Creación de un prisma triangular

```
func _add_triangle(st: SurfaceTool, a: Vector3, b: Vector3, c: Vector3)
                                                    -> void:
        # Funcion para crear el triangulo a-b-c en st
        var normal = ((c - a).cross(b - a)).normalized()
        st.set_normal(normal)
        st.add_vertex(a)
        st.add_vertex(b)
        st.add_vertex(c)
```

Transformaciones geométricas

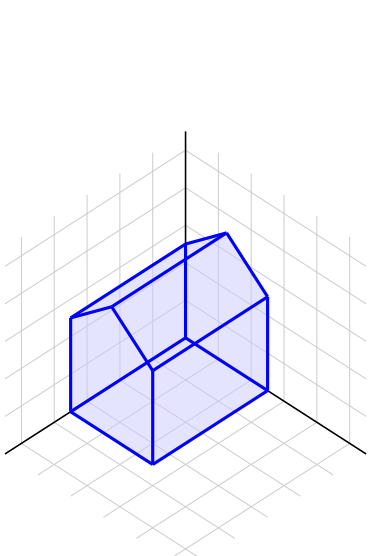
Las transformaciones geométricas son funciones que modifican las coordenadas de los puntos. Existen varias transformaciones geométricas simples que son muy útiles en Informática Gráfica para la definición de escenas y animaciones:

- **Traslación:** desplazar todos los puntos del espacio de igual forma, es decir, en la misma dirección y la misma distancia.
- **Escalado:** estrechar o alargar las figuras en una o varias direcciones.
- **Rotación:** rotar los puntos un ángulo en torno a un eje

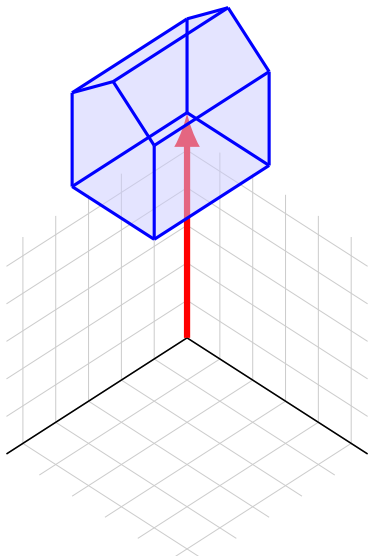
Transformación de traslación en 3D

Si $\mathbf{d} = (d_x, d_y, d_z)$ es una tupla cualquier de $\mathbb{E}_3$, la transformación de **traslación** $T_{\mathbf{d}}$ en $\mathbb{E}_3$ es una transformación geométrica que desplaza cualquier punto $\mathbf{p} = (x, y, z)$ sumándole $\mathbf{d}$, es decir:

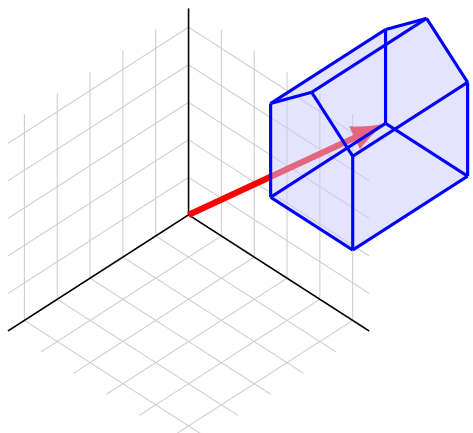
$$T_{\mathbf{d}}(\mathbf{p}) \equiv \mathbf{p} + \mathbf{d} = (x, y, z) + (d_x, d_y, d_z) = (x + d_x, y + d_y, z + d_z)$$



$\mathbf{d} = (0, 0, 0)$



$\mathbf{d} = (0, 1.2, 0)$

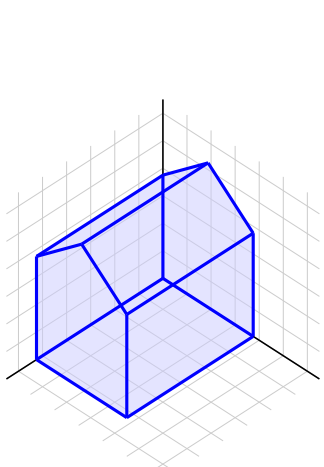


$\mathbf{d} = (0.7, 0.6, -0.5)$

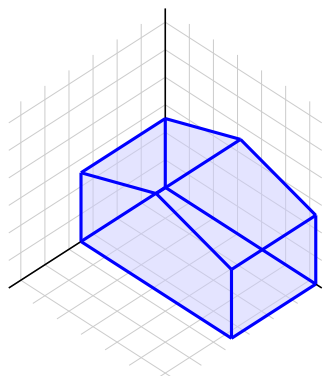
Transformación de escalado en 3D

Si $\mathbf{s} = (s_x, s_y, s_z)$, entonces una transformación de **escalado** $E_{\mathbf{s}}$ en $\mathbb{E}_3$ es una transformación geométrica que usa multiplicación componente a componente por $\mathbf{s}$, es decir para cualquier punto $\mathbf{p} = (x, y, z)$ se cumple:

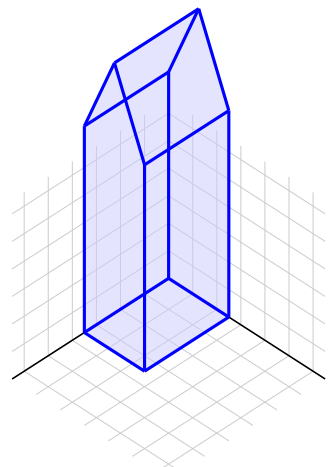
$$E_{\mathbf{s}}(\mathbf{p}) \equiv (s_x x, s_y y, s_z z)$$



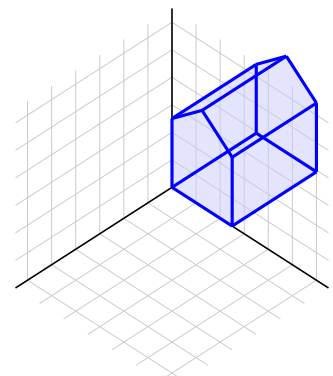
$\mathbf{s} = (1.5, 1.5, 1.5)$
uniforme
$s_x = s_y = s_z$



$\mathbf{s} = (2.5, 1, 1)$
no uniforme
$s_x \neq s_y = s_z$



$\mathbf{s} = (1, 3, 1)$
no uniforme
$s_y \neq s_x = s_z$

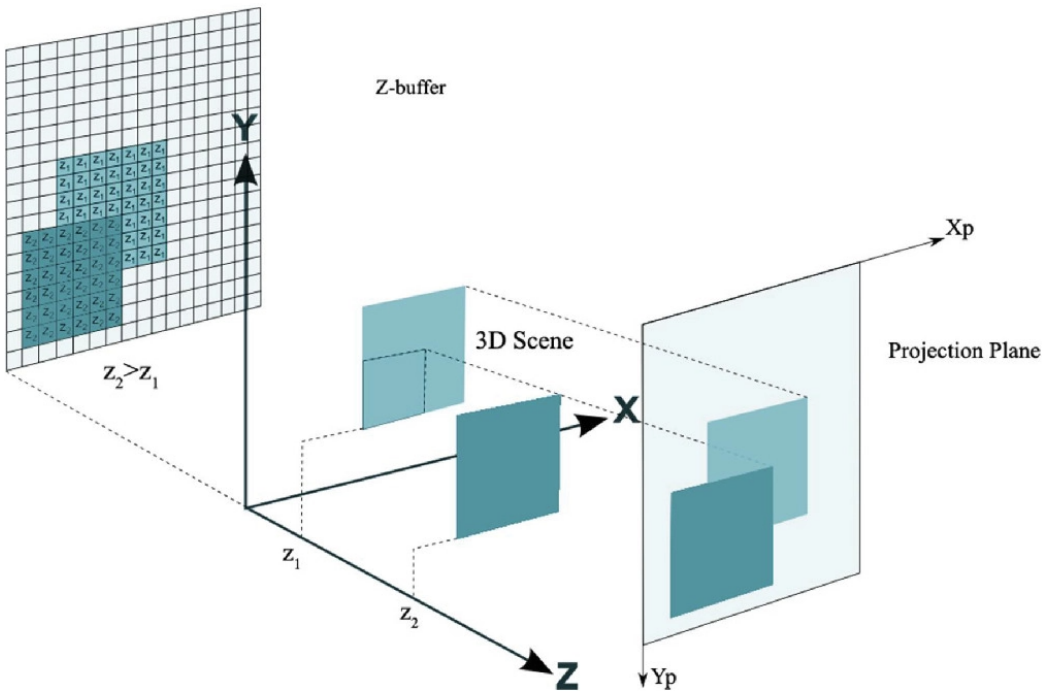


$\mathbf{s} = (1, 1, -1)$
espejo
$s_z < 0$

Z-Buffer

El algoritmo de Z-Buffer es el método usado para eliminar las partes ocultas

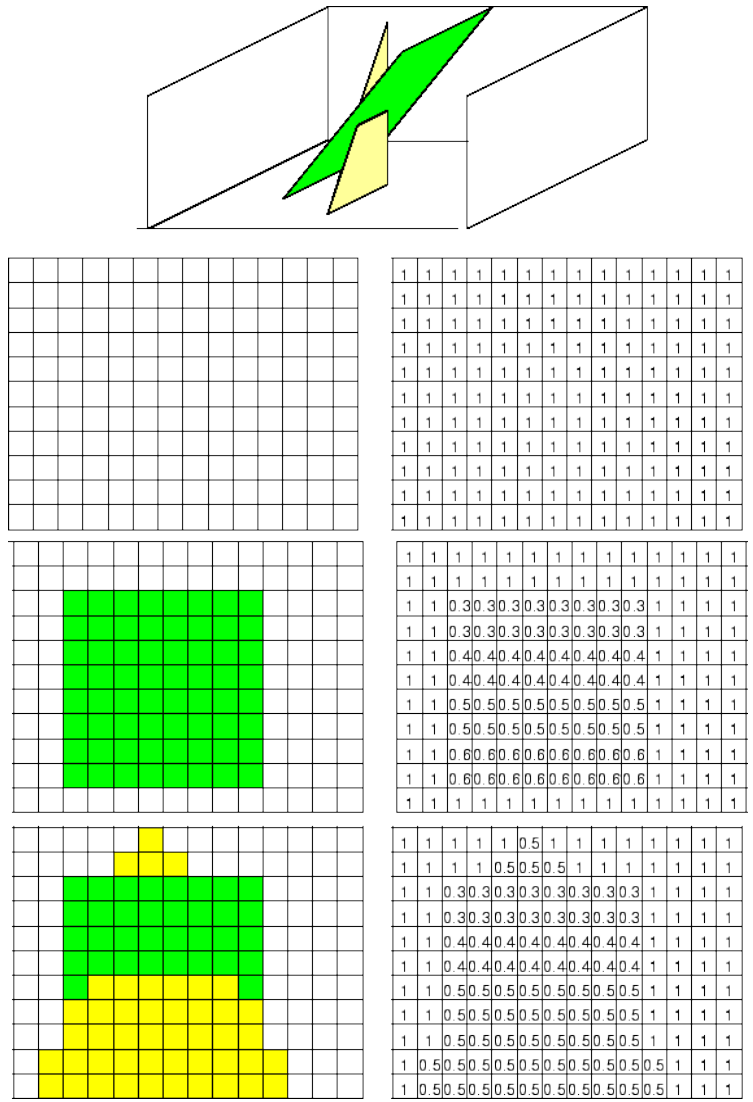
Para cada pixel que se va a dibujar se calcula su profundidad respecto a la cámara y solo se dibuja si no se ha dibujado previamente un objeto mas próximo en ese pixel.



https://docs.hektorprofe.net/graficos-3d/24-profundidad-con-z-buffer/

Z-Buffer

```
color canvas[resx,resy]
int zbuffer[resx,resy]=Zfar
for each primitive
  Rasterizar
    for each fragment
      if zbuffer[x,y] > fragment.depth
        zbuffer[x,y] = fragment.depth
        canvas[x,y] = fragment.color
```



http://glasnost.itcarlow.ie/

Z-Buffer

Las profundidades se transforman a un intervalo

$$z' = \frac{Zfar + Znear - 2\frac{Zfar\ Znear}{Z}}{Zfar - Znear}$$

y se discretizan (para almacenarlas como enteros sin signo).

